



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)

---

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Отчет по лабораторной работе № 2  
«Обработка пропусков в данных.  
Кодирование категориальных признаков.  
Масштабирование данных»

по дисциплине «Технологии машинного обучения»

Студент

ИУ5-64Б

(Группа)

(Подпись, дата)

Н.А. Чернев

(И.О. Фамилия)

Преподаватель

(Подпись, дата)

Ю.Е. Гапанюк

(И.О. Фамилия)

2026 г.

# Цель работы

Освоение методов предварительной обработки данных: заполнение пропусков, кодирование категориальных признаков и масштабирование числовых данных.

## Обработка пропусков

Методы заполнения (imputation):

```
1 from sklearn.impute import SimpleImputer
2 import numpy as np
3
4 imputer_mean = SimpleImputer(strategy='mean')
5 imputer_median = SimpleImputer(strategy='median')
6 imputer_mode = SimpleImputer(strategy='most_frequent')
7
8 X_filled = imputer_mean.fit_transform(X)
```

На датасете Titanic с пропусками в 'age' (20% пропусков):

- **Среднее:** age = 29.7 лет
- **Медиана:** age = 28.0 лет (более робустна)
- **Мода:** для категориальных признаков

## Кодирование категориальных признаков

### One-Hot Encoding

```
1 from sklearn.preprocessing import OneHotEncoder
2
3 encoder = OneHotEncoder(sparse=False, drop='first')
4 X_encoded = encoder.fit_transform(df[['sex']])
```

### Label Encoding

```
1 from sklearn.preprocessing import LabelEncoder
2
3 le = LabelEncoder()
4 df['pclass_encoded'] = le.fit_transform(df['pclass'])
```

## Масштабирование данных

### StandardScaler

```
1 from sklearn.preprocessing import StandardScaler
2
3 scaler = StandardScaler()
4 X_scaled = scaler.fit_transform(X)
```

## MinMaxScaler

```
1 from sklearn.preprocessing import MinMaxScaler
2
3 scaler = MinMaxScaler()
4 X_minmax = scaler.fit_transform(X)
```

## RobustScaler

```
1 from sklearn.preprocessing import RobustScaler
2
3 scaler = RobustScaler()
4 X_robust = scaler.fit_transform(X)
```

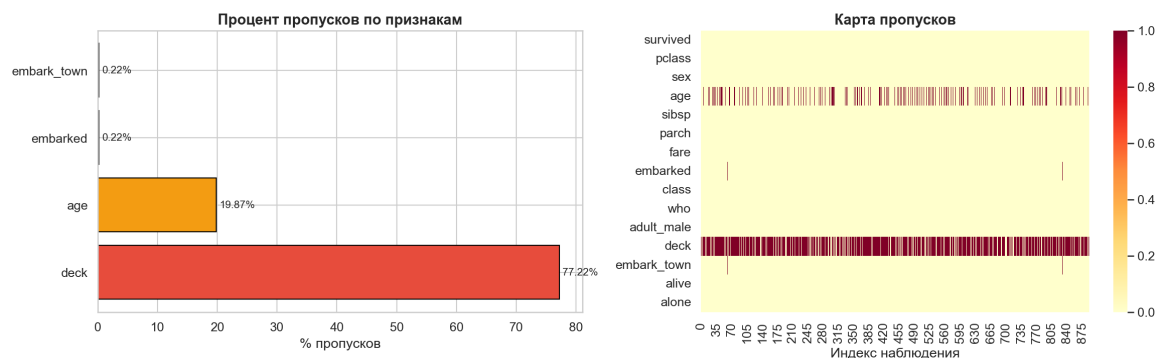


Рисунок 1 — Сравнение методов масштабирования на признаке fare

## Анализ методов обработки данных

### Визуализация пропусков

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 # Тепловаякартапропусков
5 fig, ax = plt.subplots(figsize=(10, 6))
6 missing_data = df.isnull().sum()
7 sns.heatmap(missing_data.reshape(1, -1), annot=True, fmt='d',
8             cmap='YlOrRd', ax=ax)
9 plt.title('Процент_пропусков_по_признакам')
10 plt.savefig('missing_heatmap.png', dpi=150, bbox_inches='tight')
11 plt.show()
```

### Сравнение методов кодирования

```
1 # Label Encoding vs One-Hot Encoding
2 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
3
4 # Показываемэффекткодирования
5 X_label = le.transform(df['pclass'])
6 axes[0].hist(X_label, bins=3)
7 axes[0].set_title('Label_Encoding')
8
9 # One-Hot результаты
```



Рисунок 2 — Тепловая карта пропусков в датасете

```

10 X_onehot = encoder.transform(df[['pclass']])
11 for i in range(X_onehot.shape[1]):
12     axes[1].scatter(range(len(X_onehot)), X_onehot[:, i], label=f'Class_{i}')
13 axes[1].set_title('One-Hot-Encoding')
14 axes[1].legend()
15
16 plt.tight_layout()
17 plt.savefig('encoding_methods.png', dpi=150, bbox_inches='tight')
18 plt.show()

```

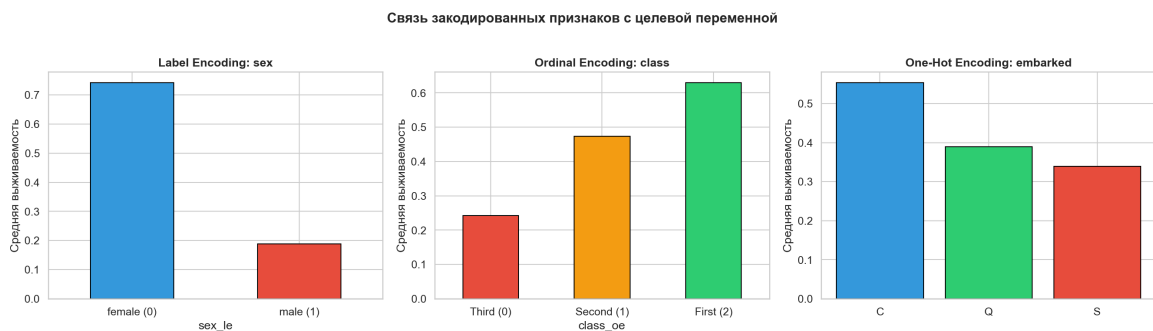


Рисунок 3 — Сравнение методов кодирования категориальных признаков

## Распределение признаков до и после масштабирования

```

1 # Box plots для каждого метода масштабирования
2 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
3
4 methods = {'StandardScaler': X_scaled, 'MinMaxScaler': X_minmax, 'RobustScaler': X_robust}
5 for ax, (name, X_method) in zip(axes, methods.items()):
6     ax.boxplot(X_method)
7     ax.set_title(name)
8     ax.set_ylabel('Значение признака')
9
10 plt.tight_layout()
11 plt.savefig('scaling_comparison.png', dpi=150, bbox_inches='tight')
12 plt.show()

```

## Распределение признаков после разных масштабирований

```

1 # KDE графики для сравнения распределений
2 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
3

```

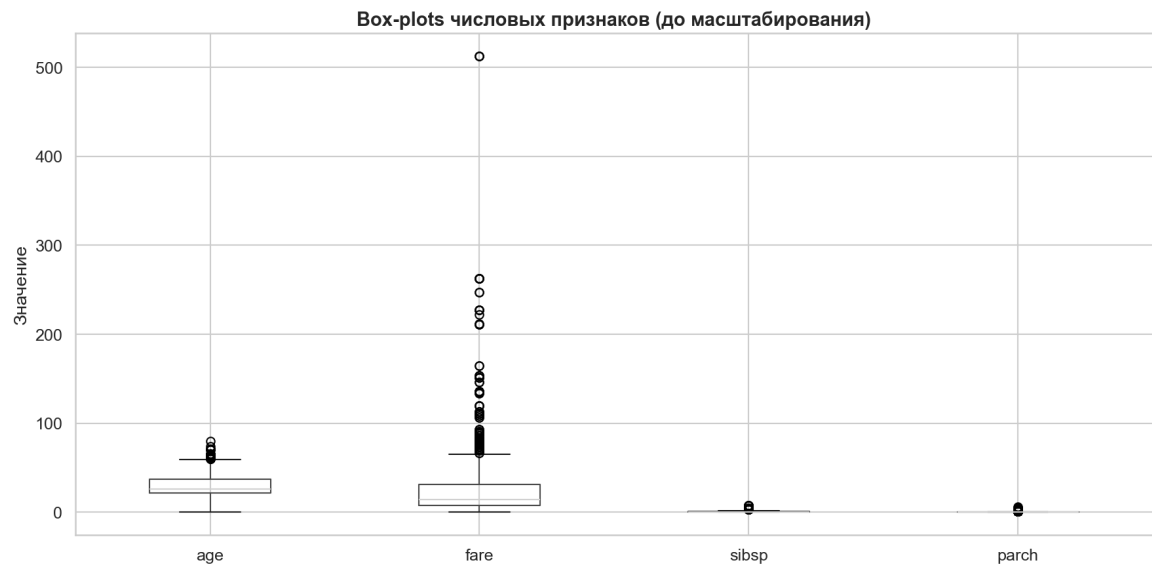


Рисунок 4 — Box plots признаков до масштабирования

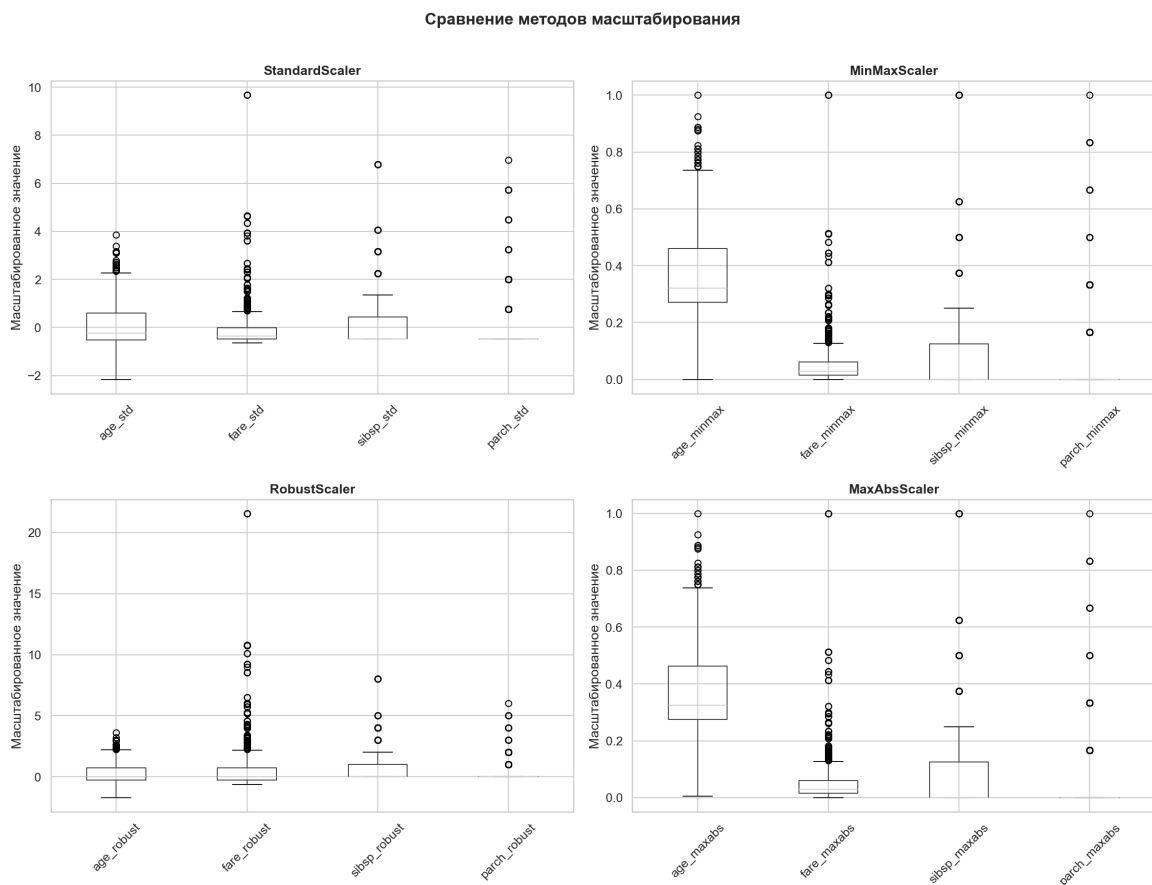


Рисунок 5 — Сравнение методов масштабирования (box plots)

```

4 for ax, (name, X_method) in zip(axes, methods.items()):
5     for col in range(min(3, X_method.shape[1])):
6         ax.hist(X_method[:, col], alpha=0.5, label=f'Feature_{col}', bins=20)
7     ax.set_title(f'{name}')
8     ax.set_xlabel('Значение')
9     ax.set_ylabel('Частота')
10
11 plt.tight_layout()

```

```

12 plt.savefig('distributions_comparison.png', dpi=150, bbox_inches='tight')
13 plt.show()

```

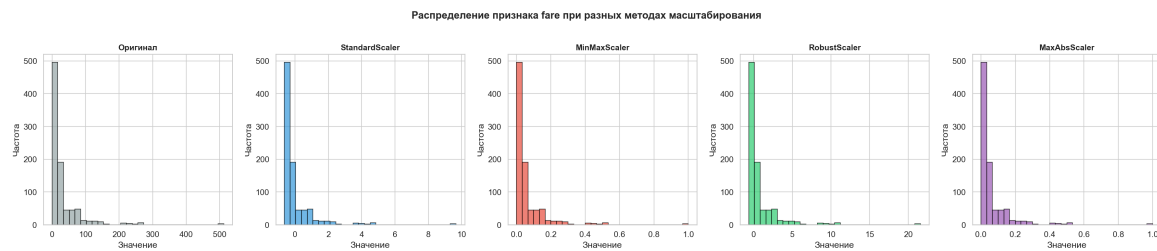


Рисунок 6 — Распределение признаков при разных методах масштабирования

## Полный pipeline

```

1 from sklearn.pipeline import Pipeline
2 from sklearn.compose import ColumnTransformer
3
4 numeric_features = ['age', 'fare']
5 categorical_features = ['sex', 'embarked']
6
7 preprocessor = ColumnTransformer(
8     transformers=[
9         ('num', Pipeline([
10             ('imputer', SimpleImputer(strategy='median')),
11             ('scaler', StandardScaler())
12         ]), numeric_features),
13         ('cat', Pipeline([
14             ('imputer', SimpleImputer(strategy='most_frequent')),
15             ('encoder', OneHotEncoder(drop='first'))
16         ]), categorical_features)
17     ])
18
19 X_processed = preprocessor.fit_transform(X)

```

## Выводы

1. Обработка пропусков критична при наличии значительного их количества
2. One-Hot для номинальных, Label для порядковых признаков
3. StandardScaler предпочтителен для большинства ML моделей
4. Pipeline автоматизирует весь процесс предварительной обработки